Domain Consolidation

Owner	Luke Capizano, Meredith Knott
Department	Engineering, Marketing, Product
Status	In Progress, Ready for GTM, Released
Stakeholders	Trevor Hinesley, Dave Kart, AJ Robertson, Geoff, Jess Hoffman
Project Lead	Dylan Price
Target Release	January 5, 2026 → March 6, 2026
Strategic Themes	Accelerate music discovery & workflows, Reach new markets & partners, Improve Free > Paid CVR, Reduce Tech Debt, Operational Requirements
Asana Project	app.asana.com/1/4...878808
Team Size	Empty
Plan Type	Product Spec
People Required	Empty
Time Budget	Empty
Changelogs	Website & App Update | Domain Consolidation
Slack Channel	Empty
Status 1	Not started

21 more properties

Comments

Add a comment...

Create a new page and select `Product Spec` from the list of template options to automatically generate the format below.

Approvals
Overview
Success Criteria
User Stories
Scope
Additional Resources
Designs
Stakeholder Review
Related Documents
Follow Up Tasks
Implementation Plan - Technical Architecture
SEO Consolidation: robots.txt & Sitemap Strategy
Production Preview Strategy for Domain Consolidation
Domain Consolidation — 100% Rollout Plan

Approvals

	Approver(s)	✅ Approved	Approved Date
Test Spec	*Approver 1*		
	Approver 2		
Design			
Copy			
Metrics			

Overview

Problem Statement

A brief description of the problem we're trying to solve. Why is this valuable to work on?

Soundstripe currently operates across two primary domains:

- **Marketing / Content:** `www.soundstripe.com`
- **App Experience:** `app.soundstripe.com`

This split introduces:

- Fragmented user experience
- Inconsistent tracking across tools
- Increased technical overhead (cookies, sessions, auth, analytics)
- Complications for experimentation, onboarding, and personalization
- Decreased SEO impact

Proposed Work

A high-level overview of what we're building and why it will solve the problem.

To improve usability, analytics accuracy, and long-term scalability, we will consolidate domains into a **single, cohesive domain experience**.

- Provide a seamless user experience across marketing and application flows
- Eliminate cross-domain tracking and session issues
- Ensure continuity for analytics, user education, and behavioral tools
- Minimize SEO, performance, and security risks

Success Criteria

What criteria must be met in order to consider this project a success?

- No increase in login or checkout drop-off post-launch
- ≥ 95% continuity in user/session tracking across tools
- No loss of SEO traffic after migration (±5%)
- Zero P0 incidents related to auth, billing, or content access

- Long Term Goal:
  - 20% overall increase in traffic from organic search (3-6 months post-launch)
  - U/X clarity for users navigating between sites

AJ Robertson Jan 5
And Statsig?

AJ Robertson Jan 5

And ad platforms? I d
will require much othe

User Stories

How should the product behave from a user's perspective?

Users

Prospective User (Anonymous Visitor)

- Browses marketing or app pages
- Evaluates pricing and catalog
- Searches and listens to music
- Signs up for an account

2. Authenticated Customer (Free Account or Subscriber)

- Logs in
- Searches and licenses music, sound effects and video
- Purchases transactional licenses
- Manages account and billing

3. Internal Stakeholders

- Marketing: funnels, attribution
- Product: onboarding, engagement
- Data: analytics accuracy
- Support: reduced user confusion

Scope

Requirements

What requirements should this project fulfill?

- Domain strategy decision (primary domain)
- Architecture planning outline for DBS to review prior to starting development
- Redirects and routing between old and new URLs

  www.soundstripe.com would be fronted by fastly which would provide your reverse proxy/CDN:

  - Default route → HubSpot origin
  - /app/* route → App origin (SPA) via path-based routing
- Authentication, cookies, and session handling
- Analytics and tracking updates
- Tool reconfiguration (Mixpanel, Userpilot, Clarity, Statsig, etc)
- QA, rollout, and monitoring

Navigation & Experience

Requirement: Users should experience a single, uninterrupted journey regardless of where they start.

User Stories

- As a visitor, I can move from marketing pages to sign-up and into the app without noticing a domain change.
- As a logged-in user, I remain authenticated as I move between content and application features.
- As a returning user, my bookmarks continue to work or redirect correctly.

Authentication & Sessions

Requirement: Authentication must persist across the consolidated domain.

User Stories

- As a user, I only need to log in once per session.
- As a user, I am not unexpectedly logged out during navigation.
- As a user, my account, subscription, and licenses remain intact after consolidation.

Analytics & Tracking (User-Facing Impact)

Requirement: User actions should be consistently tracked across the entire experience.

User Stories

- As a user, onboarding experiences feel personalized and relevant.
- As a user, educational tooltips trigger at the right time and place.
- As a user, support teams can accurately diagnose issues based on session replays.

J Joe Barnett Jan 29

We should add "meta
"internal links" as wel
references the URL (e
twitter:url or any <a h

J Joe Barnett Jan 29

SEO & Accessibility

Requirement: Existing content must remain discoverable and accessible.

User Stories

- As a user coming from search, I land on the correct page without errors.
- As a user, links shared externally continue to work.

Technical Requirements

Domain Strategy

- Select a **single canonical domain** (e.g., `soundstripe.com`)
- All secondary domains must:
  - 301 redirect to the canonical domain
  - Preserve path, query parameters, and hash fragments

Routing & Redirects

- Implement server-side 301 redirects
- Ensure:
  - No redirect chains
  - Query strings preserved
  - Backward compatibility for deep links
- Set up a server alert for any 404 error reports

Authentication & Cookies

- Cookies must:
  - Be scoped to the top-level domain (`.soundstripe.com`)
  - Use `SameSite=None; Secure` where required
- Session storage and JWTs must be compatible with the new domain
- OAuth callbacks and auth providers updated accordingly

Analytics & Tooling

Mixpanel

- Single project or consistent `distinct_id` across all surfaces
- Ensure:
  - User identity persists through login/signup
  - Events are not double-fired post-redirect
- Update tracking initialization to the canonical domain
- Validate funnels pre- and post-launch

Userpilot

- Reconfigure installation to:
  - Load on the canonical domain
  - Maintain user identity and segmentation

We can also add a tas
Console Site Change
the app subdomain th

J Joe Barnett Jan 29

https://developers.go

J Joe Barnett Jan 29

We (Digital Brand Ser
supply a "master crav
generate that has any
that have gotten at le
impression in search
last 16 months, or has

- Validate:
  - Onboarding flows still trigger correctly
  - No duplicate user profiles

Microsoft Clarity

- Update domain settings to capture:
  - Session replays across the full experience
- Confirm:
  - No session breaks during redirects
  - PII masking remains intact

SEO & Metadata

- Update:
  - Canonical tags
  - Sitemap URLs
  - Robots.txt if needed
- Implement:
  - 301 redirects from all old URLs
  - HSTS updates where applicable
- Monitor Google Search Console for crawl errors

Performance & Reliability

- Redirect latency < 100ms
- No user-noticeable increase in:
  - Page load latency
- Rollout plan must include:
  - Extensive QA and testing
    - Test all of our apps, admin, extensions, and scan our blog posts, landing pages, etc., and look for broken links
      - Test sitemap in Staging
      - Use Fastly to test otherwise
    - Set up a server alert for any 404 error reports
  - Rollback strategy

Security & Compliance

- TLS certificates valid for all domains/subdomains
- No mixed content issues
- Validate:
  - CSRF protections
  - CORS configurations
- Ensure compliance with privacy and consent frameworks

Future Work

List requirements that we know we want to do, but will do later.

- List

Non-Requirements

Include anything related to this project that is out of scope.

- Visual redesign of marketing or app UI
- Feature changes unrelated to domain behavior
- Major backend refactors beyond what's required

Additional Resources

Primary Repositories:

- https://github.com/SoundstripeEngineering/agentic-search - This is our AI Search (or Supe) feature within app.soundstripe.com used for searching for music. This is a lovable.dev project this is rendered within an iframe on the https://app.soundstripe.com/royalty-free-music page. Click this "AI Search" button to open the UI.
- https://github.com/SoundstripeEngineering/hubspot - This houses some, but not all, the code on www.soundstripe.com. It contains scripts that are included in the `<head>` plus so extra code related to our Statsig and Mixpanel integrations. Cookies and URLs *may* need updating.

Extensions:

- https://github.com/SoundstripeEngineering/twitch-extension - Twitch Extension
- https://github.com/SoundstripeEngineering/adobe-ppro-extension - Adobe Premier Pro Extension
- https://github.com/SoundstripeEngineering/adobe-express - Adobe Express Extension

Mobile App:

- https://github.com/SoundstripeEngineering/mobile - This is our React Native mobile app that Xavier and others from DBP have worked on.

Designs

Include designs as necessary. These can also be included in the user stories or requirements sections.

Embed Figma

Stakeholder Review

Include any relevant stakeholders input needed or note why none is required.

See Approvals

Related Documents

Include any links to relevant external documents.

Domain Consolidation

Follow Up Tasks

What needs to be done next for this proposal?

- [ ] Sync with UX Engineers to design XYZ
- [ ] Create tickets
- [ ] ...

Implementation Plan - Technical Architecture

> **Note:** This document complements the existing Domain Consolidation overview. It provides the technical implementation details for DBS to review.

> **Updated Feb 10:** Plan updated with staging validation results. Phases 1-2 validated in staging. Phase 3-4 in progress.

Summary

This plan details **how** we will implement the domain consolidation from a Fastly/VCL perspective, including:
- HubSpot proxy setup
- **Two-layer routing** preserving Project Overtake (React-to-Rails migration)
- 301 redirect implementation
- SEO asset consolidation
- Rails & React app prefix handling
- Phased rollout strategy

Alignment with Success Criteria:
- No loss of SEO traffic (±5%) ✓
- 20% organic increase (3-6 months) ✓
- Zero P0 incidents ✓
- ≥95% tracking continuity ✓

Current State Analysis

Domain Architecture (Today)

Domain	Destination	CDN
`www.soundstripe.com`	HubSpot (direct)	HubSpot CDN
`app.soundstripe.com`	Fastly → App backends	Fastly
`api.soundstripe.com`	Fastly → API	Fastly
`metrics.soundstripe.com`	Fastly → Mixpanel	Fastly
`ab.soundstripe.com`	Fastly → Statsig	Fastly

Application Stacks (3 Total)

Stack	Host	Purpose	Migration Status
React SPA	Netlify	Main web app	Being phased out
Full-stack Rails	Cloud66 (DigitalOcean)	New app + admin + APIs	Growing (target)
Elixir/Phoenix	DigitalOcean	Legacy API (`/v1/*`)	Maintenance

Project Overtake (Active Migration)

> ⚠️ **Critical System:** "Project Overtake" is Soundstripe's active React-to-Rails migration strategy. The domain consolidation **must preserve this routing logic**.

What it is: A Fastly-managed routing system that serves specific pages from the Rails app instead of the React SPA.

How it works: The `Page Migrations` dictionary in Fastly maps specific URLs to the Rails backend. When a route is ready to migrate, it's added to this dictionary with value `"1"`.

Why it's critical: This is the core mechanism enabling Soundstripe's gradual migration from React to Rails. Breaking this would disrupt active development work.

Current VCL Routing (app.soundstripe.com)

```
Priority 1: Specific Rails assets → Rails Priority 2: Routes in Page Migrat
ions dictionary (value="1") → Rails ← PROJECT OVERTAKE Priority 3: /new/* p
aths → Rails Priority 4: Assets with /new/ referer → Rails Priority 5: Ever
ything else → React SPA (default)
```

Note: Elixir/Phoenix routes (`/v1/*`) handled in api.soundstripe.com service.

Target State Architecture

Domain Strategy

Canonical Domain: `soundstripe.com`

Domain	Behavior
`soundstripe.com`	**Primary** - All traffic
`www.soundstripe.com`	301 → `soundstripe.com`
`app.soundstripe.com`	301 → `soundstripe.com/app/*`

Routing Architecture (Two-Layer Routing)

```
soundstripe.com │ ├─ LAYER 1: Domain Router (NEW) │ │ │ ├─ 301 Redirects (w
ww → apex, app subdomain → /app) │ │ │ ├─ SEO assets (/robots.txt, /sitema
p.xml) → Fastly synthetic │ │ │ ├─ Logged-in root (/) with authToken cookie
→ 302 → /app │ │ │ ├─ Bare app paths (/account/*, /sign_in, /sign_out) → 30
1 → /app/* │ │ │ ├─ Marketing paths (/, /blog, /about, etc.) → HubSpot back
end │ │ │ ├─ App paths (/app/*) │ │ └─ → LAYER 2 (Project Overtake routing)
│ │ │ └─ API paths (/v1/*) → Elixir/Phoenix │ └─ LAYER 2: Project Overtake
(EXISTING - preserved) │ ├─ Routes in Page Migrations dictionary → Rails ├─
/new/* paths → Rails └─ Default → React SPA
```

Implementation Phases

Timeline Summary

Phase	Duration	Hours	Status
Phase 1: HubSpot Proxy	~1 week	24-32h	✅ Validated in staging
Phase 2: Domain Consolidation	1 week	32-40h	✅ Validated in staging
Phase 3: Tool Reconfiguration	0.5 week	16-24h	⏳ In progress
Phase 4: Testing & QA	~1 week	24-32h	⏳ In progress
Phase 5: Phased Rollout	0.5-1 week	16-24h	❌ Pending (production)
Total Duration	**4-5 weeks**	**112-152h**	

Phase 1: HubSpot Proxy Setup ✅ VALIDATED IN STAGING

Duration: ~1 week (24-32h)

Objective: Route marketing content through Fastly to HubSpot

Staging Results

- [x] ~~HubSpot backend (`F_hubspot_production`) created and working~~
- [x] ~~Marketing pages render correctly through Fastly~~
- [x] ~~HubSpot assets (CSS, JS, images, fonts) load correctly~~
- [x] ~~No mixed content warnings~~

For Production

- [ ] Add `www.soundstripe.com` to the Fastly service
- [ ] Configure SSL certificate
- [ ] DNS change: `www.soundstripe.com` → Fastly
- [ ] Verify HubSpot tracking codes fire
- [ ] Verify forms submit successfully

Phase 2: Domain Consolidation ✅ VALIDATED IN STAGING

Duration: 1 week (32-40h)

Objective: Unify traffic under soundstripe.com

Staging Results

Routing (validated with curl + browser testing):
- [x] `/` → HubSpot (200)
- [x] `/app/*` → React SPA (200)
- [x] `/app/account/profile` → Rails via Project Overtake (302/200)
- [x] `/new/*` → Rails (200)

301 Redirects:
- [x] `staging-mkt.soundstripe.com/*` → 301 → `staging.soundstripe.com/*`
- [x] Query parameters preserved in redirects
- [x] No redirect chains or loops

Logged-in redirect:
- [x] Root `/` with `authToken` cookie → 302 → `/app`
- [x] Root `/` without cookie → HubSpot (200)

Bare path redirects (safety net for Rails internal links):
- [x] `/account/*` → 301 → `/app/account/*`
- [x] `/sign_in` → 301 → `/app/sign_in`
- [x] `/sign_out` → 301 → `/app/sign_out`

SEO Assets:
- [x] `/robots.txt` → Unified (HubSpot + App rules) via Fastly synthetic
- [x] `/sitemap.xml` → Sitemap index pointing to both sub-sitemaps
- [x] `/sitemap-marketing.xml` → Proxy to HubSpot's sitemap
- [x] App sitemap URLs include `/app` prefix (credential updated, sitemap regenerated)

Authentication:
- [x] Cookie `authToken` (domain `.soundstripe.com`) works across React and Rails
- [x] Rails reads `authToken` via `Authentication` concern
- [x] Auth redirects include `/app` prefix (via SCRIPT_NAME)

App Code Changes (deployed to staging)

React App (branch: `feature/domain-consolidation-prefix`):

File	Change
`app/app.js`	Dynamic `basename` for react-router (`/app` when served under `/app`)
`app/utils/basePath.js`	**NEW** — `getBasePath()` + `prependBasePath()` for `window.location.href` calls
`app/utils/routeConfiguration.js`	`prependBasePath()` on navigation
`app/components/Link/New/index.js`	`prependBasePath()` on Rails links
`app/hocs/withRouter.js`	`prependBasePath()` on redirects
`app/components/TabbedPage/index.js`	`prependBasePath()` on tab navigation
`app/containers/App/index.js`	TurboFrameWrapper conditional (dev only)

Rails App:

File	Change
`app/controllers/concerns/app_prefix.rb`	**NEW** — Sets `SCRIPT_NAME` + provides `app_path()` helper
`app/controllers/web/base_controller.rb`	`include AppPrefix`
`app/views/shared/_navigation.html.erb`	~30 hardcoded links → `app_path()`
`app/views/web/playlists/music/show.html.erb`	Breadcrumb link → `app_path()`
`app/views/web/account/profiles/show.html.erb`	"See Plans" button → `app_path()`
`app/views/web/sessions/new.html.erb`	Sign Up / Forgot Password → `app_path()`

How AppPrefix works:
- Fastly sends `X-App-Request: 1` header for `/app/*` requests
- Rails concern detects this header and sets `SCRIPT_NAME = "/app"`
- All route helpers automatically include `/app` prefix
- `app_path()` helper handles hardcoded paths in views
- Without the header (direct Rails access), everything works as before

VCL Changes Summary

Three VCL snippets implemented in staging:

1. `vcl_recv`: Inverted routing (Layer 1 + Layer 2), bare path redirects, Rails asset priority, SEO intercepts, 301 redirects, logged-in redirect
2. `vcl_deliver`: Location header rewriting (adds `/app` to Rails redirect responses)
3. `vcl_error`: Handlers for synthetic responses (robots.txt, sitemap, redirects)

For Production

- [ ] Add `soundstripe.com` (apex) to Fastly service
- [ ] Configure SSL (A record to Fastly IPs)
- [ ] Implement `app.soundstripe.com/*` → 301 → `soundstripe.com/app/*`
- [ ] Implement `www.soundstripe.com/*` → 301 → `soundstripe.com/*`
- [ ] Update sitemap credential to `https://soundstripe.com/app`
- [ ] Update `sitemaps_host` in `config/sitemap.rb` to `https://soundstripe.com`
- [ ] Regenerate sitemap with `rake sitemap:refresh`
- [ ] Update robots.txt domain references
- [ ] Update sitemap index domain references

Phase 3: Tool Reconfiguration ⏳ IN PROGRESS

Duration: 0.5 week (16-24h)

Objective: Update all tracking and analytics tools

Mixpanel

Already proxied via `metrics.soundstripe.com`
- [ ] Verify `distinct_id` persists across domain change
- [ ] Update domain references in hubspot repo
- [ ] Validate events not double-fired
- [ ] Test funnels pre/post-launch

Userpilot

- [ ] Update installation for canonical domain
- [ ] Verify user identity maintained
- [ ] Test onboarding flows
- [ ] Check for duplicate profiles

Microsoft Clarity

- [ ] Update domain settings
- [ ] Verify session replays work across redirects
- [ ] Confirm PII masking intact

Statsig

Already proxied via `ab.soundstripe.com`
- [ ] Verify experiments continue working
- [ ] Update any domain-specific configurations

Google Analytics / Search Console

- [ ] Add new property for `soundstripe.com`

- [ ] Submit updated sitemap
- [ ] Monitor crawl errors

Phase 4: Testing & QA ⏳ IN PROGRESS

Duration: ~1 week (24-32h)

Objective: Comprehensive validation before rollout

Functional Testing

Project Overtake Validation: ✅ Validated in staging
- [x] Routes in Page Migrations dictionary route to Rails under `/app`
- [x] `/new/*` paths still route to Rails
- [x] Default app routes still route to React SPA
- [x] No interference between Layer 1 and Layer 2 routing

App Routes: ✅ Validated in staging
- [x] Auth flows work (login/logout)
- [x] Session persistence across navigation
- [x] Rails navigation links include `/app` prefix
- [x] React navigation to Rails includes `/app` prefix

Marketing Routes: ✅ Validated in staging
- [x] HubSpot pages render through Fastly
- [x] Note: HubSpot CTAs link to production (expected — staging uses prod HubSpot as backend)

Extensions & Mobile: ❌ Not yet tested
- [ ] Twitch Extension - API calls work
- [ ] Adobe Premiere Pro - Auth works
- [ ] Adobe Express - Auth works
- [ ] Mobile App - API endpoints work

AI Search (Agentic): ❌ Not yet tested
- [ ] agentic-search iframe works
- [ ] Search functionality on `/royalty-free-music` page

SEO Validation (with DBS)

- [x] ~~301 redirects verified~~
- [ ] Canonical URLs correct
- [x] ~~robots.txt accessible and unified~~
- [x] ~~Sitemap valid (index + sub-sitemaps)~~
- [ ] Google URL Inspection tool checks (production only)

Security Checklist

- [x] ~~TLS certificates valid (staging verified)~~
- [ ] No mixed content issues (full browser audit pending)
- [ ] CSRF protections working
- [ ] CORS configurations correct

Phase 5: Phased Rollout ❌ PENDING (Production)

Duration: 0.5-1 week (16-24h)

Objective: Safe production deployment

Rollout Stages

Stage	Traffic	Duration	Criteria
1	5%	24h	No 5xx errors, redirects work
2	25%	24h	Analytics tracking verified
3	50%	48h	No user complaints
4	100%	-	Full rollout

Monitoring

Metrics to Watch:
- Error rates (target: no increase from baseline)
- Redirect latency (target: < 100ms)
- Page load times (target: no increase)
- Login/checkout drop-off (target: no increase)

Alert Triggers:
- 5xx error rate > 1%
- 4xx error rate > 10%
- Auth failures spike
- Support ticket surge

Rollback Plan

1. Deactivate current VCL version in Fastly (< 5 min)
2. Activate previous version
3. If DNS changed: revert DNS (longer propagation)
4. If Rails changes deployed: revert branch (redeploy)

Success Validation (at 100% rollout)

- [ ] No increase in login/checkout drop-off
- [ ] ≥95% tracking continuity verified
- [ ] SEO traffic within ±5%
- [ ] Zero P0 incidents

Technical Approach

Redirects

- 301 redirects implemented at Fastly edge
- Target latency: < 100ms
- Query parameters and URL fragments preserved
- No redirect chains
- Bare app path catch-all prevents broken links from Rails internal navigation

HubSpot Routing

- Marketing paths routed to HubSpot backend via Fastly
- App paths maintain existing routing logic (page_migration table)
- Root path (`/`) behavior based on authentication state:
  - Anonymous users → HubSpot (marketing homepage)
  - Authenticated users → 302 → `/app` (detected via `authToken` cookie)

Cookies & Sessions

- Cookie scope: `.soundstripe.com` (works across all subdomains)
- `authToken` JWT cookie shared between React and Rails
- Rails verifies via `Authentication` concern (`Session::Authenticate`)
- React sets via `js-cookie` with `domain: ".soundstripe.com"` , `secure: true` , `sameSite: "lax"`

App Prefix Handling

- **React**: Dynamic `basename` in react-router + `prependBasePath()` utility for `window.location.href`
- **Rails**: `AppPrefix` concern sets `SCRIPT_NAME` from `X-App-Request` header; `app_path()` helper for hardcoded view paths
- **VCL**: Location header rewriting in `vcl_deliver` (safety net for any missed Rails redirects)

SEO Assets

- Unified `robots.txt` served from Fastly edge (synthetic response)
- Sitemap index pointing to marketing sitemap (HubSpot) and app sitemap (S3)
- App sitemap generated by Rails `sitemap_generator` gem, uploaded to S3
- App sitemap URLs use `soundstripe.com/app/*` format (controlled by `soundstripe_app_url` credential)

Project Overtake Integration

> **Critical:** This section ensures the active migration system remains functional.

Implementation Strategy:
- Domain routing (Layer 1) added **above** existing VCL logic
- Project Overtake routing (Layer 2) remains **completely unchanged**
- `/app/*` requests strip prefix then pass through to existing Rails/React routing
- Page Migrations dictionary continues to control which routes go to Rails

What changed:
- New VCL snippets for Layer 1 routing (HubSpot, redirects, SEO)
- `X-App-Request` header marks `/app/*` requests for downstream processing
- `vcl_deliver` rewrites Location headers for `/app/*` responses
- React `basename` and `prependBasePath()` for client-side navigation
- Rails `AppPrefix` concern for server-side URL generation

What stayed the same:
- Page Migrations dictionary logic
- Rails/React backend selection within Layer 2
- `/new/*` path handling
- Asset routing with referer checks

Risk Assessment

Risk	Likelihood	Impact	Mitigation	Status
Project Overtake disruption	Low	**Critical**	Layer 2 routing unchanged; validated in staging	✅ Mitigated
HubSpot tracking breaks	Medium	High	Tested through Fastly proxy in staging	⏳ Needs prod verification
Auth issues post-redirect	Medium	High	Cookie sharing validated; AppPrefix handles Rails URLs	✅ Mitigated
SEO ranking drop	Low	High	Phased rollout, daily monitoring	❌ Pending rollout
Extension/mobile app breaks	Low	Medium	Pre-launch testing needed	⏳ Not yet tested
VCL complexity increases	Medium	Medium	Clear layer separation; documented in SESSION_HANDOFF.md	✅ Mitigated
Cross-env cookie deletion	Low	Medium	Fixed hardcoded `app.soundstripe.com` links in Rails views	✅ Mitigated

Dependencies & Coordination

Access Required

- [x] ~~Fastly admin access~~
- [x] ~~DNS management (DNSimple)~~
- [x] ~~Cloud66 (Rails deploy + rake tasks)~~
- [ ] Google Search Console (coordinate with DBS)
- [ ] HubSpot (for updating CTAs/links post-consolidation)

Repositories Updated

- [x] ~~app - React basename, prependBasePath, TurboFrame fix~~
- [x] ~~api - AppPrefix concern, navigation links, session links, sitemap credential~~

Repositories to Verify

- [ ] hubspot - URL/cookie updates
- [ ] agentic-search - Verify iframe works
- [ ] Extensions (Twitch, Adobe PPro, Adobe Express) - Verify auth works
- [ ] mobile - Verify API endpoints

Staging DNS Reference

```
staging.soundstripe.com → Fastly (consolidated domain) staging-mkt.soundstr
ipe.com → Fastly (redirect to staging) staging-react.soundstripe.com → Netl
ify (React app direct) staging-web.soundstripe.com → Cloud66 (Rails app dir
ect) staging-api.soundstripe.com → Fastly (API)
```

Questions Resolved

- ✅ **API subdomain:** `api.soundstripe.com` stays separate (out of scope)
- ✅ **Root path:** Anonymous → HubSpot; Authenticated → 302 → /app
- ✅ **Path structure:** App routes use `/app/*` prefix
- ✅ **Auth mechanism:** `authToken` JWT cookie on `.soundstripe.com` shared between React and Rails
- ✅ **Rails URL generation:** `AppPrefix` concern with `SCRIPT_NAME` + `app_path()` helper
- ✅ **SEO approach:** Unified robots.txt (Fastly synthetic) + sitemap index (standard approach)

SEO Consolidation: robots.txt & Sitemap Strategy

Context

After domain consolidation, `soundstripe.com` will serve both marketing (HubSpot) and app content under `/app/*`. Today these live on separate domains with separate SEO configurations. We need to unify them.

Andres Castillo Feb 2

@Joe Barnett just up
considering search e
now

Current State

Asset	Marketing (HubSpot)	App
robots.txt	`www.soundstripe.com/robots.txt` — blocks HubSpot internal paths	`app.soundstripe.com/robots.txt` — blocks private routes (/account, /checkout, etc.)
Sitemap	`www.soundstripe.com/sitemap.xml` — auto-generated by HubSpot	Generated by Rails (`sitemap_generator` gem), hosted on S3 at `/assets/sitemaps/sitemap.xml.gz`

Consolidated Approach

robots.txt — Synthetic response from Fastly

We intercept `/robots.txt` at the Fastly layer and serve a unified file that combines rules from both sources. This is the cleanest option — no dependency on either backend, fully under our control, and easy to update.

The unified file will:
- Keep HubSpot's rules for their internal paths (`/_hcms/`, `/hs/`, etc.)
- Add app-specific disallows with the new `/app` prefix (`/app/account`, `/app/checkout`, etc.)
- Block `/new/` (internal Rails/Turbo routes)
- Reference the consolidated sitemap index

Sitemap — Index file pointing to both sources

Rather than merging sitemaps into one massive file, we'll use a **sitemap index** — the standard approach for sites with multiple content sources. Google handles these natively.

The setup:

```
/sitemap.xml → Sitemap index (served by Fastly, synthetic) Points to both s
ub-sitemaps below /sitemap-marketing.xml → Proxied to HubSpot's original si
temap Contains: blog posts, landing pages, marketing content /assets/sitema
ps/ → App sitemap on S3 (already working via Netlify proxy) sitemap.xml.gz
Contains: songs, artists, sound effects, playlists, tags
```

This way each source manages its own sitemap independently. HubSpot auto-generates theirs, Rails regenerates via `rake sitemap:refresh`.

Implementation (all in Fastly VCL)

Three routes to handle:

1. `/robots.txt` → Fastly serves synthetic response
2. `/sitemap.xml` → Fastly serves synthetic sitemap index XML
3. `/sitemap-marketing.xml` → Fastly proxies to HubSpot's original `/sitemap.xml`

No changes needed in Rails, React, or HubSpot for this piece. Everything lives in VCL.

Production consideration: App sitemap URLs

The app sitemap is generated by Rails (`config/sitemap.rb`) and currently produces URLs like `app.soundstripe.com/browse/music`. After consolidation, these need to become `soundstripe.com/app/browse/music`. This is a config change in `sitemap.rb` + a regeneration (`rake sitemap:refresh`), and should be done as part of the production cutover.

Staging Validation

All of the above is being validated in staging first:
- `staging.soundstripe.com/robots.txt` → unified robots
- `staging.soundstripe.com/sitemap.xml` → sitemap index
- `staging.soundstripe.com/sitemap-marketing.xml` → HubSpot sitemap
- `staging.soundstripe.com/assets/sitemaps/sitemap.xml.gz` → app sitemap (already accessible)

Production Preview Strategy for Domain Consolidation

The Problem

We need to validate the domain consolidation in production before exposing it to the general public. The previous attempt to go live directly led to issues that required a rollback. We want to avoid that by letting the internal team test on real production infrastructure first.

The Solution: Cookie-Based Feature Flag in Fastly VCL

Fastly VCL controls all routing decisions before any request reaches a backend. We can leverage this to branch the entire routing logic based on a simple cookie, enabling the consolidated experience only for specific people.

For users without the cookie: Everything works exactly as it does today. Zero changes to the user experience.

For users with the cookie: They see the full domain consolidation experience — `/library` prefix routing, redirects, the works — on real production data and infrastructure.

Implementation Phases

Phase A: Test `/library` Routing on Production (No DNS Changes)

`app.soundstripe.com` already goes through Fastly. We can deploy the cookie-based routing logic to the existing production VCL **without changing any DNS records**.

What we deploy:
- A Fastly Edge Dictionary (`dc_whitelist`) containing secret tokens for each tester
- VCL logic that checks for a `dc_preview` cookie at the start of every request
- If the cookie token matches the whitelist → apply the new `/library` prefix routing
- If no cookie or token doesn't match → existing routing, completely untouched

What the team can test:
- `app.soundstripe.com/library/royalty-free-music` — React app under `/library` prefix
- `app.soundstripe.com/library/account/profile` — Rails pages under `/library` prefix
- Navigation between React and Rails pages
- Authentication flows (login, logout, session persistence)
- Asset loading (JS, CSS, fonts)
- Tracking tools (Mixpanel, Google Analytics, etc.)
- AI Search (agentic iframe communication)

What normal users see: Absolutely nothing different. The cookie check is the very first thing in the VCL — if it's not there, the request follows the exact same path it always has.

Risk level: Very low. The VCL change is additive (new code paths gated behind a cookie check). The existing code paths are not modified.

How testers activate it: A simple bookmarklet or manual cookie in DevTools:

```
Cookie name: dc_preview Cookie value: (personal secret token from the white
list) Domain: .soundstripe.com Secure: yes
```

How to add/remove testers: Edit the Fastly Edge Dictionary. No deploy needed. Changes take effect in seconds.

Phase B: DNS Migration (Transparent to Users)

Only after the team has validated Phase A, we tackle the DNS change — which is the step that caused issues previously.

Preparation (days before the DNS change):
1. Lower the TTL for `www.soundstripe.com` DNS record to 60 seconds
2. Add `soundstripe.com` and `www.soundstripe.com` as domains in the Fastly service
3. Provision and verify TLS certificates in Fastly for these domains
4. Test the HubSpot proxy by curling Fastly directly with the appropriate Host header

The DNS change:
- Point `www.soundstripe.com` → Fastly (`dualssl.ssl.global.fastly.net` or appropriate endpoint)
- VCL behavior: **ALL traffic from www → proxied to HubSpot**. No consolidated routing yet. This is purely a transparent infrastructure change.

If anything breaks: DNS rollback takes ~60 seconds (because we lowered the TTL beforehand).

Success criteria: HubSpot pages load identically through Fastly as they did directly. No visual differences, no broken assets, no errors.

Phase C: Enable Full Consolidated Experience for Team

Once DNS is stable (1-2 days of monitoring):

- Whitelisted users now see the complete experience:
  - `soundstripe.com/library/*` → App (React/Rails)
  - `soundstripe.com/*` → HubSpot (marketing, regardless of auth state)
  - `app.soundstripe.com/*` → 301 redirect → `soundstripe.com/library/*`
  - All 301 redirects, robots.txt, sitemaps, etc.
- Normal users still see everything as before

What the team validates:
- Full end-to-end experience on the final domain (`soundstripe.com`)
- HubSpot pages served through Fastly
- Marketing → App transitions
- SEO elements (canonical URLs, structured data, sitemaps)
- Email/notification links (if HOSTS constant is updated)
- OAuth flows (Frame.io, Apple Sign-In)

Phase D: Gradual Public Rollout

Transition from cookie-based to percentage-based:

Stage	Audience	Duration	Monitoring
1	Whitelist only	1-2 days	Team validates everything
2	Whitelist + 5% of traffic	1 day	Watch error rates, SEO metrics
3	25% of traffic	1 day	Watch analytics continuity
4	50% → 100% of traffic	1 day	Full rollout

Fastly's `randombool()` function handles percentage-based rollout natively. Combined with the whitelist, the VCL logic would be:

```
if (whitelisted) → consolidated experience else if (search engine bot) → cu
rrent experience (bots excluded until 100%) else if (randombool(X, 100)) →
consolidated experience else → current experience
```

Search engine crawlers (Googlebot, Bingbot, etc.) are explicitly excluded from the percentage rollout. They only see the consolidated experience once we reach 100% and remove the feature flag. This ensures consistent indexing — crawlers never see a mix of old and new URL structures.

At any point during the rollout, we can revert instantly by setting the percentage to 0. Only whitelisted users would continue seeing the new experience.

How the Cookie Mechanism Works

Setting Up the Whitelist

In Fastly, we create an Edge Dictionary called `dc_whitelist`:

Key (Token)	Value	Person
`a1b2c3d4e5`	`1`	Luke
`f6g7h8i9j0`	`1`	Dev Team
`k1l2m3n4o5`	`1`	QA

Tokens are random strings. Adding or removing entries takes effect in seconds without any deploy.

VCL Logic (Simplified)

```
# First thing in request handling — check for preview cookie Is dc_preview
cookie present? ├── YES → Is token in dc_whitelist? │ ├── YES → Apply conso
lidated routing │ └── NO → Normal routing (ignore invalid token) └── NO → N
ormal routing (nothing changes)
```

Activating Preview

Each tester gets a personal token and sets it as a cookie. Options:

1. **Bookmarklet** (easiest) — one click to activate/deactivate
2. **DevTools** — Application → Cookies → Add manually
3. **Internal admin page** — A simple page that sets the cookie (if preferred)

The cookie is set on `.soundstripe.com` domain, so it works across `app.soundstripe.com`, `soundstripe.com`, and `www.soundstripe.com`.

Deactivating Preview

Delete the `dc_preview` cookie, or remove the token from the Edge Dictionary.

Risk Assessment

Phase	Risk Level	Rollback Time	What Could Go Wrong
A	Very Low	Seconds (revert VCL)	VCL syntax error (caught by Fastly validation)
B	Medium	~60 seconds (DNS revert)	HubSpot proxy misconfiguration, TLS issues
C	Low	Seconds (revert VCL)	Routing edge cases for whitelisted users only
D	Low-Medium	Seconds (set % to 0)	Unexpected behavior at scale

Key safety property: At no point does a failure affect users who don't have the preview cookie. The feature flag is fail-closed — if anything goes wrong with the cookie check, the request falls through to normal routing.

Timeline Estimate

Phase	Duration	Dependencies
A: Cookie-based preview on app.soundstripe.com	1 day to implement, 1-2 days team testing	VCL changes only
B: DNS migration	1 day prep + execution, 1 day monitoring	TLS certs, DNS access
C: Full consolidated preview	Same VCL (already deployed), 1-2 days team testing	Phase B stable
D: Gradual rollout	3-4 days (5% → 25% → 50% → 100%)	Phase C validated

Total: ~1.5-2 weeks from start to 100% rollout. All routing logic and code changes have already been developed and validated in staging, so implementation is mostly VCL configuration and verification.

What This Means for the Team

- **No more "big bang" deployment.** The consolidation rolls out gradually with full control.
- **Real production testing.** Not staging, not a preview domain — actual production with real data, real users (for testers), real everything.
- **Instant rollback at every phase.** Cookie flag off = back to normal, immediately.
- **The DNS change (the scary part) happens after routing is already validated** on production, not before.
- **Anyone can be added or removed as a tester** in seconds, without code changes.

Staging Validation Results

The cookie-based preview mechanism has been implemented and validated in staging (last updated Feb 18, 2026):

Test	Without cookie	With cookie
Root `/`	✅ Netlify (React app, original behavior)	✅ HubSpot (marketing site)
Root `/` with `authToken`	✅ Netlify (no redirect)	✅ HubSpot (no redirect — auth state doesn't affect routing)
`/library/royalty-free-music`	—	✅ Netlify via /library routing
`/account/profile`	✅ Rails direct (302 → sign_in)	✅ 301 → /library/account/profile
`/robots.txt`	✅ Original Netlify robots.txt	✅ Unified synthetic robots.txt (both sitemaps referenced directly)

Both paths are fully isolated. The feature flag works as designed.

Why 100% Instead of Gradual Rollout

During the initial 5% rollout attempt, we discovered that Fastly's `randombool()` function evaluates randomly on **every request**, not per user session. This caused:

- Users selected into the 5% were redirected to `www.soundstripe.com/library`, but subsequent requests (the redirect itself, JS/CSS assets) had a 95% chance of **not** being selected — routing them to HubSpot instead of the app, resulting in 404s and broken loading screens.

The consolidated routing logic itself **has been working correctly for weeks** through preview cookie testing (Phase C). The issue was purely with the random selection mechanism. Going to 100% is simpler — no `randombool`, no sticky cookies, no per-request inconsistency — and carries no additional risk since rollback is instantaneous.

What Changes for Users

Domain	Before	After

Domain Consolidation — 100% Rollout Plan

> Consolidating `app.soundstripe.com` and `www.soundstripe.com` under a single primary domain: `www.soundstripe.com`
>
> This plan replaces the gradual percentage-based rollout (Phase D) with a direct 100% activation.

Next Steps

1. ~~Implement and test the cookie mechanism in staging~~ ✅ Done
2. Review and approve this approach
3. Deploy Phase A to production
4. Team testing period
5. Proceed through Phases B → C → D